COMPUTER ENGINEERING
QUEZON CITY

Final Project: (CALAMS) Campus Automated Locker & Asset Management System

Course Code: CPE 010 **Course Title: Data Structures and Algorithms**

PROJECT INFORMATION

This projects aims to develop a Campus Automated Locker & Asset Management System (CALAMS) using C++ to improve the organization,security, and management of locker for students belongings and campus assets.The system it will simulate the process of assigning lockers, recording assets information , and monitoring availability and managing the borrowing and returning of items.

Lockers and assets in campus premises are often considered difficult to track due to the large quantity and diversity of items in storage. Should these stored items be lost or destroyed in any means or circumstances, managing and tracking these items and belongings may involve long manual processes such as tracking the information of the most recent user of the item, finding or replacing the lost item, and contacting the owner and user for actions to resolve the liabilities associated with these items. By automating this usually manual and laborious process with CALAMS powered by optimized data structures and algorithms in C++, the process of managing and tracking the status of assets, belongings, and item availability should be faster and more efficient.

Item	Details
Project Title	(CALAMS) Campus Automated Locker and Assets
Business Process Emulated	Asset Accounting & Management System
Section / Group No.	CPE21S2 / Group 4
Instructor	Engr. Jimlord M. Quejado
Date Submitted	November 6, 2026
GitHub Repository	https://github.com/jplara-cpe2025/CPE-010-CPE21S2-FINAL-PROJECT-GROUP4-2627/

Group members, roles, and contributions are listed in **Appendix B. Team Composition**.

Intended Learning Outcomes Addressed

At the end of this activity, the students should be able to:

- Apply fundamental data structures and techniques to solve engineering-related problems through algorithm design.
- Design efficient algorithms and analyze their complexity.
- Build efficient algorithms based on selected data structures using good programming styles in C++.

Objectives

General Objective: To develop a C++ program that emulates the management process of recording, status tracking, and general management of campus assets and student lockers using optimized implementations of various data structures and algorithms to maximize functionality and performance.

Specific Objectives:

1. To arrange student records using assets using array and dynamic memory allocation.
2. To track recent borrowing and returning transactions using an array based stack
3. To Store and Organize Lockers using binary search tree
4. To analyze the time complexity of the main operations theoretically and experimentally.

1. BUSINESS PROCESS

1.1 Organization and Business Process

The Locker and Assets Management Office (LAMO) located in the TIP campus offices provides supplemental service with the OSA (Office of Student Affairs) and its maintenance rooms in managing and tracking the storage and release of student belongings stored in lockers and campus assets used in laboratory activities, campus events, and other activities involved with the LAMO. The process this system emulates is a management and tracking of student lockers and storage of school assets in the campus, which are actions including but not limited to: listing, organizing, storage, and releasing.

The personnel involved in this organization come from the OSA, tool room, and the security office, which are known as those who handle items of school property and student possessions. In this program, the pieces of information that enter in the CALAMS are the following: Student name, number, locker index, date and time, item properties, and other credentials. After the processing of the provided information, the office provides the location of the item, locker index, and time released. If borrowed, the office also provides the location of the item, the tool room index, and the time for borrowing and returning.

1.2 Current Process (As-Is)

Step	Activity	Performed By	Data Involved
1	Student gets a borrowing or locker request form from the LAMO office	Student	Name, student number, type of request
2	Student fills up the form with the item or locker needed	Student	Name, student number, item or locker, date and time
3	Students get the required signatures or approval.	Student / Authorized Personnel	Request form, Signatures
4	Student submits the form and waits for their turn	Student, LAMO staff	Completed form
5	Staff checks the logbook and inventory list to see if the asset or locker is available	LAMO / Tool room staff	Logbook, list of assets and lockers
6	Staff assigns the locker or releases the asset and writes it in the logbook	LAMO / Tool room staff	Locker or asset ID, date and time
7	Student returns the asset or gives up the locker, and staff writes it in the logbook	Student, LAMO / Tool room staff	Locker or asset ID, date and time returned

1.3 Problems in the Current Process

1. **Slow Locker/Asset Processing Time:** The current locker and asset processing for students is a long manual process that involves writing down the student credentials by hand, which results in long wait times for students in queue.
2. **Long Searches and Possibility of Losing Records:** If the storage of student request forms is unmanaged and scattered in different locations, it could take a large amount of time for searching past records for the last owner or user of a certain item that was lost or destroyed.
3. **Inability to Handle Quantities of Students in Queue:** When there are many students waiting in line for their lockers and school assets to be processed, a manual process of recording credentials, checking item/locker status, and keeping track of these records, can result in long queue times and errors in checking.
4. **Difficulty monitoring assigned lockers:** Monitoring the status of lockers across various campus buildings and floor levels requires looking for specific registrations and logs of student lockers assigned in each building to check if those are occupied or vacant.

1.4 Proposed System

#	Feature	Business Task It Supports
1	Add students requesting a locker to a waiting line, then assign or release lockers in order	Locker request management (Queue)
2	Sort and display lockers by ID or by availability	Locker listing (Sorting Algorithm)
3	Record and view recent borrowing and returning transactions	Transaction History tracking (Stack)
4	Add, search, and view campus assets in the asset inventory	Asset inventory management (Binary Search Tree)
5	Register and log in student accounts	Account management (Array and Dynamic Memory Allocation)

1.5 Scope and Delimitations

Scope

- The system covers the storing, sorting, searching, and checking of status of student lockers and assets stored in the campus.
- The system sorts the lockers according to their ID number and availability using multiple sorting algorithms.
- The system runs in a Command-Line Interface (CLI), and utilizes the data structures and algorithms introduced in this course.
- The system includes a queueing functionality for students requesting for lockers and school items to allow administrators to manage each student in the queue individually.
- The system provides information for students about the availability and status of lockers and school assets by displaying the lists of items and indicating their relevant information.
- The system keeps a history of up to 50 recent borrowing and returning transactions, which the administrator can view or remove from the Transaction History menu.

Delimitations

- The system is intended for use only by one user at a time, and does not allow multiple users to access the program at the same time.
- The system is developed and written in a CLI as its intended interface for this project, and so does not support a Graphical User Interface (GUI).
- The system does not cover web application, web interface, and other frontends. It is written exclusively in C++ with a CLI.
- The system does not use a SQL database for storing credentials, information of assets and lockers, and other values used in this project.
- The system is designed as a local and offline application, and so does not send/receive information through the internet.
- The system does not record new transactions once the history reaches 50 records, until older entries are removed.

2. DATA STRUCTURES AND ALGORITHMS APPLIED

Course topics you may choose from: arrays and dynamic memory allocation; singly, doubly, and circular linked lists; stacks (array or linked-list based); queues (array or linked-list based); linear search; binary search; hashing; sorting algorithms; general trees and binary search trees; AVL trees; tree traversal algorithms; graphs (adjacency matrix or adjacency list); breadth-first search; depth-first search.

2.1 Summary of Data Structures and Algorithms Applied

#	Data Structure / Algorithm	Where It Is Used in the Business Process	Main Operations
1	Queue (linked list)	Waiting line for students requesting for lockers	enqueue, dequeue, peek
2	Sorting Algorithm	Sorting and displaying lockers by ID and by availability	bubble sort, selection sort

3	Stack (array)	Tracks recent borrowing and returning transactions	push, pop, top
4	Binary Search Tree	Storing and organizing campus assets for the asset inventory	insert, search, delete, inorder traversal
5	Array and dynamic memory allocation	Registration and account management	insert, access, search

2.2 Queue (Linked List)

Concept: A queue is a data structure that contains the sequence of elements inserted in a First-in-First-Out (FIFO) order, where the elements inserted first are the first to be removed, and the elements inserted last are the last to be removed. By its definition, enqueueing means inserting the element into the list, and dequeueing means removing the element from the list. Because it functions as a waiting line for data elements, it is a purely linear data structure.

Application in the System: For this system, a queue is used to store the sequence of students and school personnel waiting for the locker and assets management services. Since this follows the typical First-Come-First-Serve rule of waiting lines in a school service office, using a queue is suitable for this application since it has a similar ordering with its FIFO sequence. The data stored in the queue elements are name, student number, locker index, date and time logged, and assets to obtain, borrow, or use.

The steps involved in this queueing process are as follows:

1. Entering the student name and number.
2. Selecting the following options:
    - Campus Locker
    - Borrow Item(s)
3. Enter the specific item to borrow, or campus locker to access.
4. Wait in the queue until the user is their turn.

Justification: A queue is used for this task because a waiting line is a First-Come-First-Serve sequence, which this data structure exactly matches with its FIFO model. Using stacks with its Last-In-First-Out (LIFO) arrangement, is unsuitable for a waiting line as the latest elements to enqueue are served first, which is unfair for those that queued earlier.

Operations and Complexity: [Main operations used and their time complexity, e.g., enqueue O(1), search O(log n). Full analysis goes in Section 5.]

Operation	Time Complexity	Auxiliary Space
enqueue	$O(1)$	$O(1)$
dequeue	$O(1)$	$O(1)$
requestLockerFromInput	$O(1)$	$O(1)$
displayQueue	$O(K)$	$O(1)$
processFrontQueue	$O(N)$	$O(1)$

2.3 Sorting Algorithm (Bubble Sort with Early Exit & Selection Sort)

Concept: Sorting algorithms are aimed at transforming an unordered set of elements into some monotonic order (ascending/descending according to the key of elements). Two sorting approaches are considered in CALAMS:

1. **Bubble Sort with Early Exit**: Adjacent elements are sequentially compared and swapped whenever order is not correct. A boolean variable is used to see if there was at least one swap performed during the entire outer loop; the array is sorted if the number of swaps is equal to zero and the process is terminated in the best case ($O(n)$).

2. **Selection Sort**: It involves scanning the unsorted part of the array to identify the smallest key element and performing a swapping of such key with an element that occupies the corresponding index position. Although selection sort always operates in comparisons in quadratic time, $O(n^2)$, it performs not more than $n-1$ swaps that minimize memory writings.

Application in the System: The CALAMS project implements the sorting algorithm in both student and administrator portals:

- **Sorting by Numeric ID**: Initially locker/asset IDs are not arranged in any specific order. The sorting procedure sorts the locker array in ascending order. It is an important requirement for the Member #2 Binary Search module to find lockers and records in logarithmic time: $O(\log n)$.
- **Sorting by Availability Status**: Administrators and students are able to sort the array to move all available assets to the front, providing an opportunity for users to see available lockers and assets to borrow.

Justification: As it is evident from above, sorting plays an important role in terms of human readability and efficiency of searches. Randomly displayed locker IDs cause long search times not only for students but for administrators as well. Moreover, binary search is impossible without sorting, thus, it must be done beforehand. Sorting by Bubble sort with an early exit runs in constant time for already sorted data in $O(n)$. Selection Sort makes the minimal amount of memory writes, $O(n)$.

Operations and Complexity:

Operation	Best Case	Average Case	Worst Case	Auxiliary Space
Bubble Sort (with flag)	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$

2.4 Stack (Array)

Concept: A stack is a data structure that works on the Last-In First-Out principle. That means the last item added to the stack is the one to be removed. In our program we used an array that can hold up to 50 transaction records. We also used a variable called topIndex to track the position of the most recent record. When the stack is empty, topIndex starts at -1.

The stack has three main operations: push, pop, and top. The push operation adds a transaction record to the stack. The pop operation removes the most recent record from the stack. The top operation allows the user to look at the record without removing it. The program also checks if the stack is full before adding a record and checks if the stack is empty, before removing one. This helps avoid overflow and underflow conditions.

Application in the System: In our project we use the stack to keep track of borrowing and returning transactions. The stack keeps the newest transaction on top so that the program can read the newest transaction first. Each record in the stack stores a transaction ID, a username, an asset ID, a type that's either Borrow or Return and a date and time.
Here is how it works in our system:

- The student has the option to either borrow or return an asset and must provide the necessary information.
- The program checks if the stack still has space and if the required fields are filled in.
- If the information is valid, the transaction is saved using push, together with its transaction ID and date and time.
- In Transaction History, the administrator can view the latest record, display all transaction from newest to oldest or remove the latest record using pop.

Justification: We chose a stack because our feature focuses on recent transactions. Since it follows LIFO, the administrator can easily access the latest record without going through the entire history. An array-based implementation was used because we wanted to limit the history to 50 records. However, once the stack is full, no additional transactions can be added unless a record is removed. The history is also stored only while the program is running, so the records are lost when the program closes.

Operations and Complexity: The push, pop, and top operations have a time complexity of O(1) because they only access or update the top position. Viewing all transactions takes O(n) because the program goes through the records one by one.

Operation	Time Complexity
Push	O(1)
Pop	O(1)
Top	O(1)
View all transactions	O(n)

2.5 Binary Search Tree

Concept: A binary search tree is a non-linear data structure that stores and organizes elements in nodes. Each node has at most two children, where the left child contains a value smaller than its parent, while the right child has a greater value. This allows elements to be searched, inserted, and organized efficiently.

Application in the System: The binary search tree is used for asset inventory, which can store and organize the campus asset that students may borrow.Each entry in binary search it holds assets information of students such as asset IDs, name and availability status, and assets ID serves as the key.

Justification: A Binary Search tree is a good choice for implementing asset inventory in CALAMS because it provides a more efficient way to search and organize asset IDs. A binary search tree shortens the search time by comparing the target ID with the current node

Operations and Complexity:

Operation	Time Complexity
Insert	O(logn)
Search	O(logn)

2.6 Array and Dynamic Memory Allocation

Concept: An array is a linear data structure that can hold multiple elements of the same data type in contiguous memory where each element can be accessed using an index. Dynamic memory allocation can allow us to allocate and delete memory while the program is running.

Application in the System: Arrays and dynamic memory allocation are used in the registration and account management in CALAMS. They store and manage student account information like username and password.

1. The program allocates memory for 100 student accounts using new Registration[size].
2. When a student registers, the system asks for a username and password.
3. If the username is available, the system creates a Registration object and stores it in the array.
4. When a student logs in, the system searches the array for the entered username and checks whether the password matches the stored account information.
5. When the program terminates, delete[] accounts releases the dynamically allocated memory.

Justification: Array and dynamic memory allocation is a good choice for implementing registration and account management for CALAMS because the number of students and registered accounts may change over time. This can allow the system to adjust its memory usage depending on the number of accounts stored making it more flexible and efficient.

Operations and Complexity:

Operation	Time Complexity
Insert	O(1)
Access	O(1)

Search	O(n)
Delete	O(n)

3. SYSTEM FLOWCHART

Symbol	Name	Use
Oval	Terminator	Start and End
Rectangle	Process	An action or computation, e.g., "Add patient to queue"
Parallelogram	Input / Output	Reading input or displaying output
Diamond	Decision	A yes/no question, e.g., "Is the queue empty?"
Rectangle with double sides	Predefined Process	An operation detailed in another flowchart
Arrow	Flow Line	The order of steps

3.1 Main System Flowchart

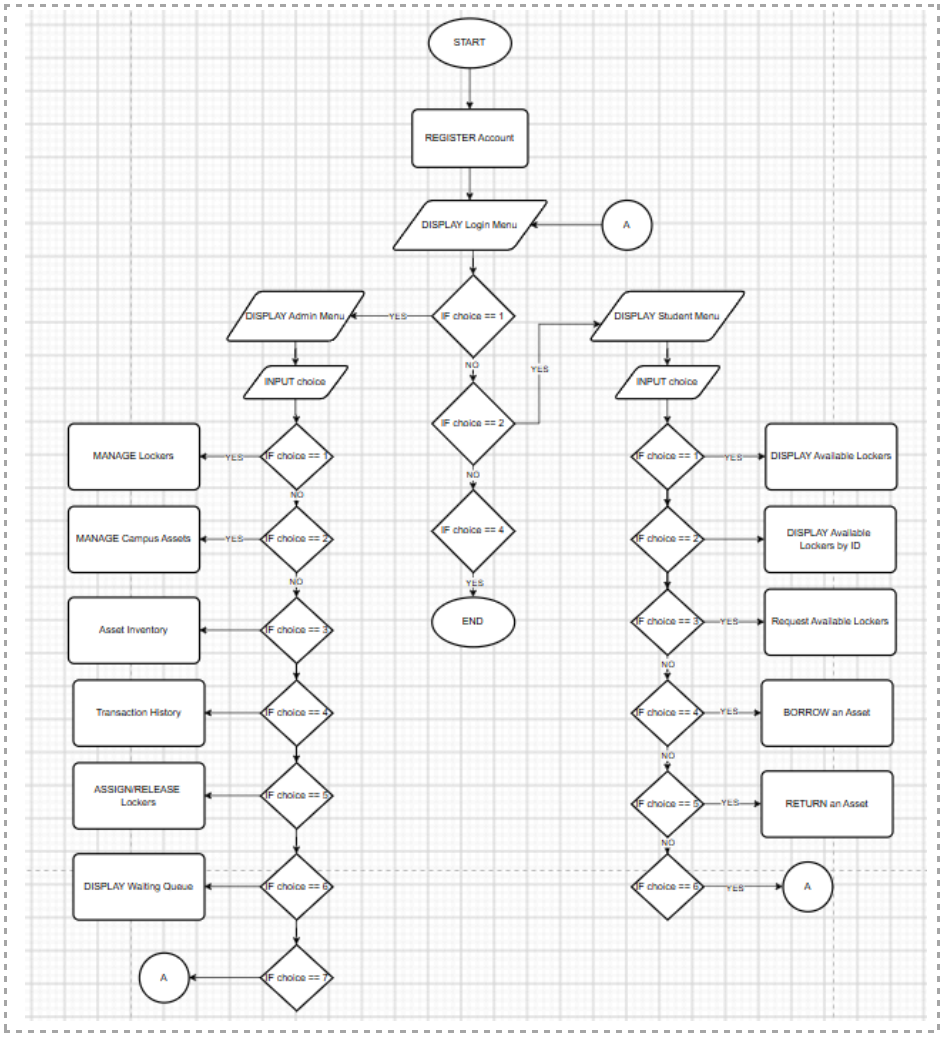


Figure 1. Flowchart of (CALAMS) Campus Automated Locker & Asset Management System

3.2 Flow Description

1. The program starts and loads the login menu which allows you to log in as either a student or administrator and register a student account.
2. If the user selects choice 1, the system displays the Admin Menu which allows administrators to manage lockers and campus assets, view the asset inventory, waiting queue, and transaction history, as well as assign or release lockers.
3. If the user selects choice 2, the system displays the Student Menu which allows students to view and request available lockers, and borrow or return assets.
4. If the user selects Choice 4, the program ends.

4. OUTPUT

Feature	Figure No.	Input Used
Start Menu	Figure 2	1, 2, 3, 4

Login & Registration Menu	Figure 3	(Admin Login) admin, admin (Registration) johnDoe, johnDoe (Student Login) johnDoe, johnDoe
[Student & Admin] Main Menu	Figure 4	1, 2, 3, 4, 5, 6, 7
Check and view available lockers	Figure 5	1, 2
Request a locker	Figure 6	3
View waiting queue	Figure 7	6
[Admin] Manage lockers and assets	Figure 8	1, 2, 3, 4
[Admin] Assign/Release locker to student (1st in queue)	Figure 9	5
[Admin] Transaction History: View most recent, view all, and remove most recent	Figure 10	4
Borrow/Return Asset	Figure 11	Borrow: 4, johnDoe, 1111 Return: 5, johnDoe, 1111
[Admin] Add, Search, Update, Display Assets	Figure 12	
Login Error Handling	Figure 13	8

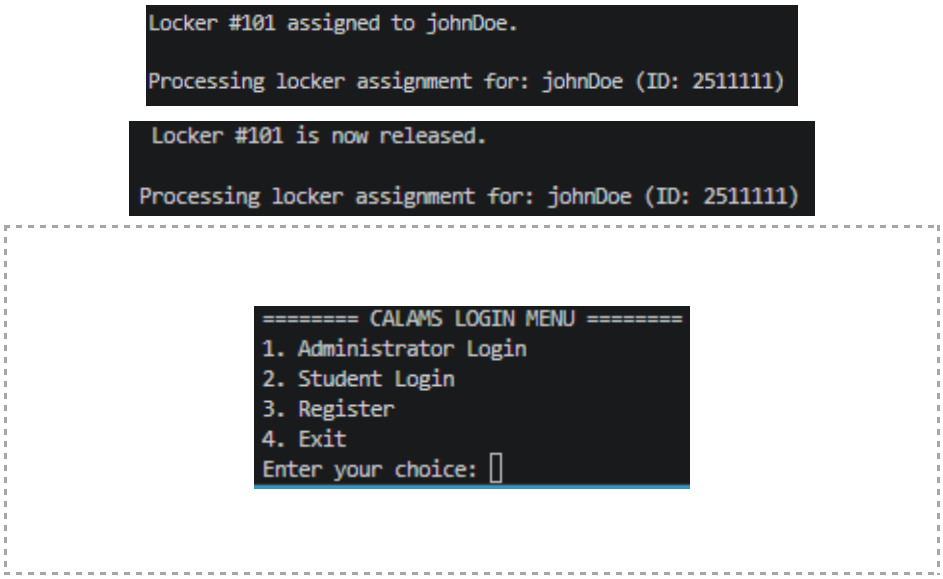


Figure 2. Start Menu - A Selection of Options for Admin Login, Student Login, Register, and Exit.



Figure 3. Login & Registration Menu - Registration, Admin, and Student Login.

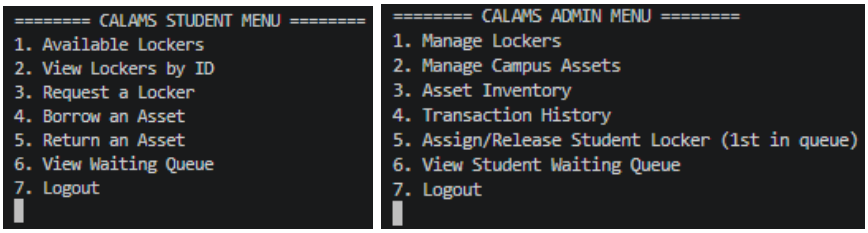


Figure 4. [Student & Admin] Main Menu - A List of Available Actions for Students and Admins.

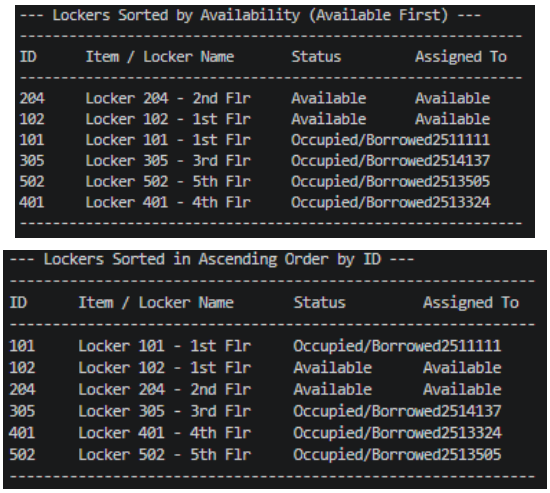


Figure 5. Check and View Available Lockers - Show lockers by availability and ID.

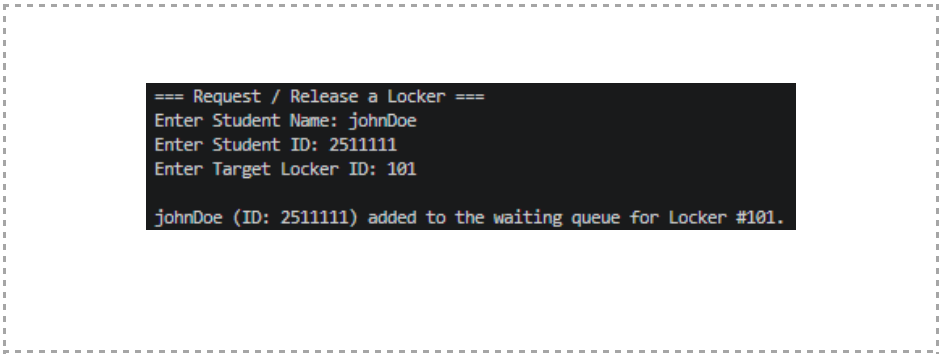


Figure 6. Request a Locker - Entering Username, Student ID, and Locker ID to Locker Queue.

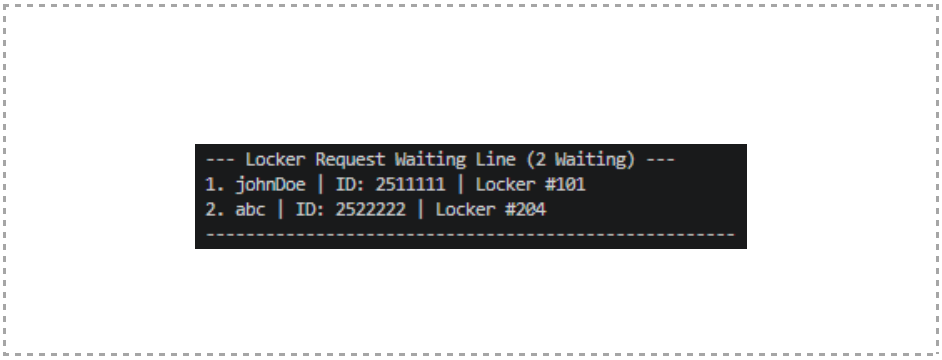


Figure 7. View Waiting Queue - Locker Request Waiting Line and Number of Students Waiting.

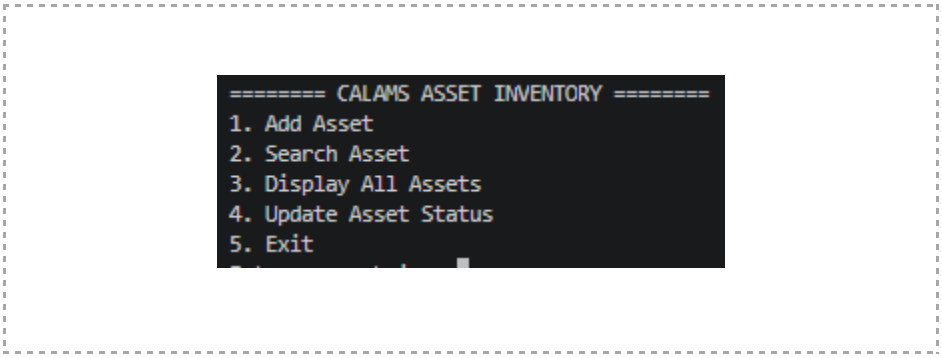


Figure 8. [Admin] Manage lockers and assets - Asset Inventory

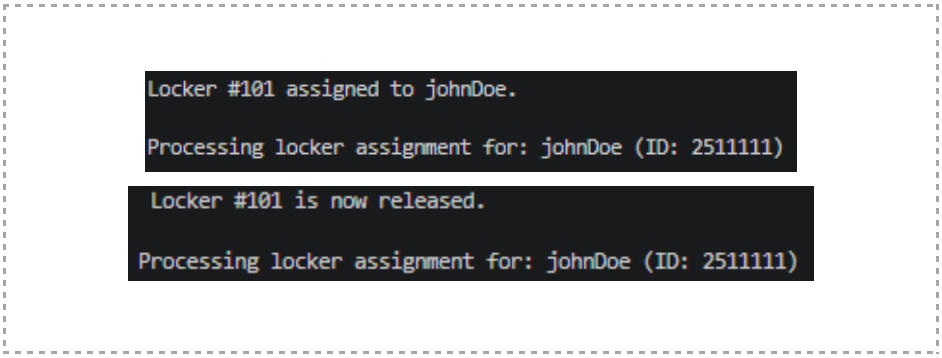


Figure 9. Locker Assignment

```
======== Transaction History ========
1. View Most Recent Transaction
2. View All Recent Transactions
3. Remove Most Recent Transaction
4. Back
Enter your choice:
```

```
Enter your choice: 1

#5 | Return | User: johnDoe | Asset: 1111 | 2026-11-06 11:10 PM
```

```
Enter your choice: 2

2/50 transactions (newest first):
#5 | Return | User: johnDoe | Asset: 1111 | 2026-11-06 11:10 PM
#3 | Borrow | User: johnDoe | Asset: 1111 | 2026-11-06 11:09 PM
```

```
Enter your choice: 3

#5 | Return | User: johnDoe | Asset: 1111 | 2026-11-06 11:10 PM
Remove this entry from the history? (y/n): y
Entry removed. The borrow/return itself is not cancelled.
```

Figure 10.[Admin] Transaction History: View most recent, view all, and remove most recent

```
Enter your choice: 4

Username: johnDoe
Asset ID: 1111
Borrow recorded.
```

```
Enter your choice: 5

Username: johnDoe
Asset ID: 1111
Return recorded.
```

Figure 11. Borrow/Return Asset

```
===== ADD ASSET =====
Enter Asset ID: 1111
Enter Asset Name: Wires
Enter Status: Working

Asset added successfully!
```

```
Enter your choice: 2
Enter Asset ID to search: 1111
===== ASSET FOUND =====
Asset ID: 1111
Asset Name: Wires
Status: Working
```

```
Enter your choice: 3

===== ASSET INVENTORY =====
1111
===== ASSET DETAILS =====

Asset ID: 1111
Asset Name: Wires
Status: Working
```

```
Enter your choice: 4
Enter Asset ID: 1111
Enter New Status:
Broken
Asset status updated successfully.
```

Figure 12. [Admin] Add, Search, Update, Display Assets

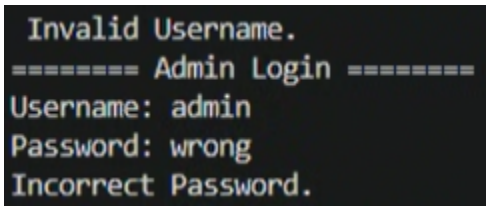


Figure 13. Login Error Handling

5. PERFORMANCE ANALYSIS

5.1 Theoretical Analysis

Operation	Data Structure / Algorithm	Best	Average	Worst	Space
Add customer to queue	Queue (Linked List)	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Search locker / asset	Queue (Linked List)	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Sort lockers by ID	Bubble Sort (with early exit)	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Sort lockers by ID	Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Sort benchmark baseline	Quick Sort / std::sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Record transaction (Push)	Stack (Array)	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Remove latest transaction (Pop)	Stack (Array)	$O(1)$	$O(1)$	$O(1)$	$O(1)$
View all transactions	Stack (Array)	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Register new account	Array + Dynamic Allocation	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Search student account	Linear Search on dynamic array	$O(1)$	$O(n)$	$O(n)$	$O(1)$

Derivation:

1. Bubble Sort with early stop flag:
   - Outer loop will execute at most $n - 1$ passes. In the worst and average case when data are randomized, inner loop will perform $(n - 1) + (n - 2) + \ldots + 1 = \frac{n(n-1)}{2}$ pairwise comparisons and swaps. Due to having n^2 polynomial term, the running time is $O(n^2)$.

- For best case with sorted locker records, the inner loop performs $n - 1$ comparisons in the first pass. With zero swaps and the help of swap boolean flag, the algorithm stops prematurely after just linear $O(n)$ work. The sorting in-place is done via swapping using temporary scalar variables with auxiliary space complexity being $O(1)$.

2. Selection Sort:
   - In Selection Sort algorithm for a given array length of n, the outer loop executes at most $n - 1$ times. On i-th pass through the array, inner loop traverses the remaining $n - i - 1$ items in order to get the minimum key position index. Irrespective of input, the sum number of comparisons is fixed: $\sum_{i=0}^{n-2}(n - i - 1) = \frac{n(n-1)}{2}$, giving $O(n^2)$ running time for best, average and worst cases.
   - However, Selection Sort performs one physical item swap on the outside loop iteration and at most $n - 1$ swaps overall, writing minimal possible memory $O(n)$ and using $O(1)$ extra space.

5.2 Experimental Analysis

Setup Item	Details
Computer (CPU, RAM)	AMD Ryzen 5600G, 16GB DDR4 3200MHz
Operating system	Windows 11 x64
Compiler and settings	g++ 16.2.0 -O2
Timing method	std::chrono::steady_clock during operation
Input data	same data record for each algorithm
Repetitions	5 runs per size and averaged

Input Size (n)	[Operation 1] Time (ms)	[Operation 2] Time (ms)	[Operation 3] Time (ms)
100	0.0702 ms	0.0439 ms	0.0036 ms
1,000	4.4467 ms	2.8410 ms	0.0415 ms
10,000	457.4845 ms	289.6485 ms	0.7744 ms
100,000	51534.4940 ms	30022.0201 ms	10.0847 ms

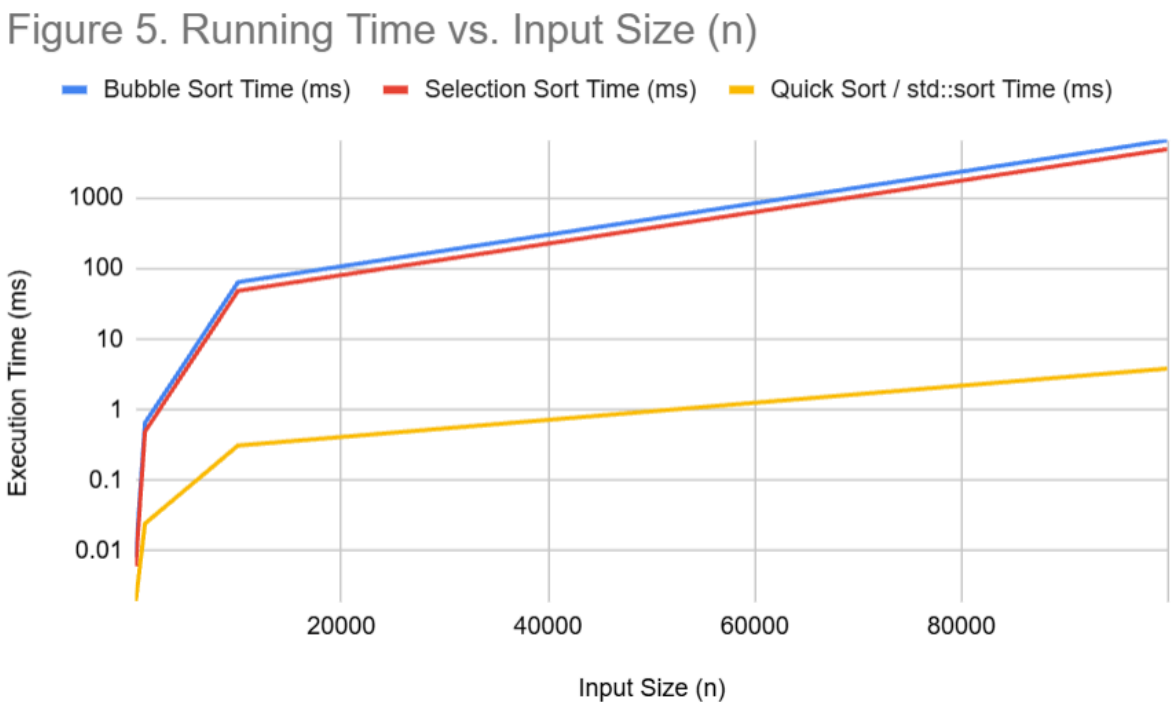


Figure 5. Running time vs. input size for Bubble Sort, Selection Sort, and Quick Sort

5.3 Comparison of Theoretical and Experimental Results

Operation	Expected Growth (Big-O)	Observed Growth	Agrees? (Yes / No)
Bubble Sort	$O(n^2)$	Scaled $\approx 102.8\times$ from $1k \rightarrow 10k$ ($4.45ms \rightarrow 457.48ms$), and $\approx 112.6\times$ from $10k \rightarrow 100k$ ($457.48ms \rightarrow 51,534.49ms$)	Yes
Selection Sort	$O(n^2)$	Scaled $\approx 101.9\times$ from $1k \rightarrow 10k$ ($2.84ms \rightarrow 289.65ms$), and $\approx 103.6\times$ from $10k \rightarrow 100k$ ($289.65ms \rightarrow 30,022.02ms$)	Yes

Quick Sort / std::sort	$O(n \log n)$	Scaled $\approx 18.6\times$ from $1k \rightarrow 10k$ ($0.0415ms \rightarrow 0.7744ms$), and $\approx 13.0\times$ from $10k \rightarrow 100k$ ($0.7744ms \rightarrow 10.0847ms$)	Yes

Discussion:

The timing results of experiments conducted on the AMD Ryzen 5600G for the test cases confirm the big-O complexity assumptions:

- **Quadratic Complexity (O(n^2)):**
  - As the input is scaled 10 times from 1,000 to 10,000, theoretically the quadratic complexity implies 10^2= 100x increase in the execution time. The experimentally observed times match this prediction: the execution time of Selection Sort increases from $2.8410ms$ to $289.6485ms$ ($\approx 101.9x$) and the Bubble Sort increases from $4.4467ms$ to $457.4845ms$ ($\approx 102.8\times$).
  - Further scaling 10 times, Selection Sort takes 30,022.0201\text{ ms}$ ($\approx 103.6\times$) and Bubble Sort takes $51,534.4940ms$ ($\approx 112.6\times$) of execution time. These algorithms both demonstrate strict adherence to the quadratic scaling behavior. Notice that Bubble Sort takes significantly more execution time compared to Selection Sort for larger n since each pass makes repeated memory exchanges while Selection Sort performs no more than $n-1$ swaps.

- **Logarithmic Linear Scaling (O(n log n)):**
  - In comparison, the Quick Sort shows high resistance to scaling: the increasing the input size from $n = 10,000$ to $n = 100,000$ results in the runtime increase by $0.7744ms$ to $10.0847ms$ ($\approx 13.0\times$), which matches the theoretical factor:

$$\frac{100000\log_2(100000)}{10000\log_2(10000)} \approx 10 \times \frac{16.61}{13.29} \approx 12.5\times$$

- **Effect of Hardware & Cache at Small Input Sizes ($n = 100$):**
  - For example, at $n = 100$, the measured execution times are fractions of the millisecond ($0.0702ms$ for Bubble Sort and $0.0036ms$ for Quick Sort). At this input size, the whole dataset fits the CPU data caches L1 & L2, and also, branching is predicted minimizing the stalls. As the input grows up to $100,000$, the memory accesses start reaching the RAM memory, making the asymptotics behavior affect the execution time.

6. CONCLUSION AND RECOMMENDATIONS

6.1 Conclusion

The **Campus Automated Locker & Assets Management System (CALAMS)** emulates the manual process of the Locker and Assets Management Office (LAMO), where students can request a forms, staff can check the logbooks for availability, and lockers and assets are assigned, released, and recorded using a hand written. CALAMS replaces these steps using the cpp command-line program. Students can register and log in using an array with dynamic memory allocation, and lockers are sorted by ID and availability using a bubble sort. Borrow and return transactions are recorded in an array-base stack that keeps the 50 most records, and the assets inventory is organized using a binary search tree. In this way, the system addresses the slow processing, long record searches que handling, and locker monitoring problems of the current process.

The performance analysis showed that the measured running time agrees with the theoretical Big - O . When the input grew from 1, 000 to 10, 000 elements, Bubble Sort is about 102.8 and the selection sort about 101.9 took roughly 100 times longer, which match in O(n log n

6.2 Recommendations

- **Switch from Elementary Sorts to Log-Linear Sorts**: Though Bubble Sort and Selection Sort can be easily programmed to be executed in place with $O(1)$ memory overhead, the resulting time complexity of $O(n^2)$ yielded disastrous execution efficiency in testing—over 51.5 seconds for $100,000$ elements. Going forward, CALAMS should employ a log-linear algorithm such as Merge Sort or Quick Sort (or std::sort) for arranging lockers and assets respectively, which will guarantee an asymptotic runtime of $O(n \log n)$ as our benchmarks have shown would sort $100,000$ elements in just $10.08ms$.
- **Replace the Linear Array Student Data Structure with a Hash Table**: The current implementation relies on linear scanning through the data structures storing students' registration and log-in information to perform user search and user login, thus having an average $O(n)$ runtime. As the student body and the number of accounts are growing throughout the institution, this average search time will be growing proportionately. The inclusion of a Hash Table (with either separate chaining or open addressing) would optimize the search process to average case $O(1)$.
- **Implementation of File Input and Output Operations**: The current version of CALAMS stores all relevant data—account records, lockers allocation, and the transaction log—with size of 50 elements—entirely in volatile memory, thus losing all state upon termination of the process. Implementing some form of file I/O (whether in binary format or through structured CSV/JSON text files) or even introducing an embedded database, such as SQLite, would provide persistence of all account and locker management data.
- **Adoption of Doubly Linked List or Dynamic Array as a Data Structure of Transactions Module**: The current transactions implementation uses a static data structure (an array-based stack) with limited size (in this case equal to 50 elements). Therefore, once reaching the maximum amount of stored information in the transaction history, any new borrow or return entry will either be dropped or rejected by the system. Using a dynamically-resizing structure, e.g. doubly linked list-based stack or a dynamic vector as a stack, would resolve the issue.

7. REFERENCES

APPENDICES

Appendix A. Source Code

A.1 main.cpp

C/C++

```
#include <iostream>
#include "Registration.h"
#include "Login.h"
#include "LockerAssetSort.h"
#include "BorrowReturnHistory.h"
#include "StudentQueue.h"
#include "AssetInventory.h"

const int TOTAL_ITEMS = 6;
LockerItem campusLockers[TOTAL_ITEMS] = {
    {305, "Locker 305 - 3rd Flr", "2514137", false},
    {101, "Locker 101 - 1st Flr", "Available", true},
    {502, "Locker 502 - 5th Flr", "2513505", false},
    {204, "Locker 204 - 2nd Flr", "Available", true},
    {102, "Locker 102 - 1st Flr", "Available", true},
    {401, "Locker 401 - 4th Flr", "2513324", false}
};
AssetInventory inventory;

void studentMenu();
void adminMenu();
void assetMenu();

int main(){
    int choice;
    do{
        std::cout << std::endl;
        std::cout << "======== CALAMS LOGIN MENU ========" << std::endl;
        std::cout << "1. Administrator Login" << std::endl;
        std::cout << "2. Student Login" << std::endl;
        std::cout << "3. Register" << std::endl;
        std::cout << "4. Exit" << std::endl;
        std::cout << "Enter your choice: ";
        std::cin >> choice;
        std::cout << std::endl;

        switch(choice){
            case 1:
                if(adminLogin()){
                    adminMenu();
                }
                break;
            case 2:
                if(studentLogin()){
                    studentMenu();
                }
                break;
            case 3:
                clearScreen();
                Register();
```

```
                break;
            case 4:
                clearScreen();
                std::cout << "Goodbye!" << std::endl;
                break;
            default:
                clearScreen();
                std::cout << "Invalid Choice." << std::endl;
                break;
        }
    }while(choice != 4);

    cleanupAccounts();
    return 0;
}

StudentQueue studentWaitingQueue;

void studentMenu(){
    int choice;
    do{
        std::cout << std::endl;
        std::cout << "======== CALAMS STUDENT MENU ========" << std::endl;
        std::cout << "1. Available Lockers" << std::endl;
        std::cout << "2. View Lockers by ID" << std::endl;
        std::cout << "3. Request a Locker" << std::endl;
        std::cout << "4. Borrow an Asset" << std::endl;
        std::cout << "5. Return an Asset" << std::endl;
        std::cout << "6. View Waiting Queue" << std::endl;
        std::cout << "7. Logout" << std::endl;
        std::cout << "Enter your choice: ";
        std::cin >> choice;;
        std::cout << std::endl;
        switch(choice){
            case 1:
                // Calls availability sorting function
                sortByAvailability(campusLockers, TOTAL_ITEMS);
                std::cout << "--- Lockers Sorted by Availability (Available
First) ---";
                displayLockerItems(campusLockers, TOTAL_ITEMS);
                break;
            case 2:
                // Calls bubble sort / selection sort by ID
                bubbleSortById(campusLockers, TOTAL_ITEMS);
                std::cout << "--- Lockers Sorted in Ascending Order by ID ---";
                displayLockerItems(campusLockers, TOTAL_ITEMS);
                break;
            case 3:
                studentWaitingQueue.requestLockerFromInput();
                break;
```

```
            case 4:
                std::cout << "[Feature under development: Borrow Asset]\n";
                transactionHistory.recordTransaction(HISTORY_BORROW);
                break;
            case 5:
                std::cout << "[Feature under development: Return Asset]\n";
                transactionHistory.recordTransaction(HISTORY_RETURN);
                break;
            case 6:
                studentWaitingQueue.displayQueue();
                break;
            case 7:
                clearScreen();
                break;
            default:
                std::cout << "Invalid Choice." << std::endl;
                break;
        }
    }while(choice != 7);
}

void adminMenu(){
    int choice;
    do{
        std::cout << std::endl;
        std::cout << "======== CALAMS ADMIN MENU ========" << std::endl;
        std::cout << "1. Manage Lockers" << std::endl;
        std::cout << "2. Manage Campus Assets" << std::endl;
        std::cout << "3. Asset Inventory" << std::endl;
        std::cout << "4. Transaction History" << std::endl;
        std::cout << "5. Assign/Release Student Locker (1st in queue)" <<
std::endl;
        std::cout << "6. View Student Waiting Queue" << std::endl;
        std::cout << "7. Logout" << std::endl;
        std::cout << "Enter your choice: ";
        std::cin >>choice;
        std::cout << std::endl;
        switch(choice){
            case 1:
                selectionSortById(campusLockers, TOTAL_ITEMS);
                std::cout << "[Admin] All Lockers Sorted Numerically via
Selection Sort:";
                displayLockerItems(campusLockers, TOTAL_ITEMS);
                break;
            case 2:
                sortByAvailability(campusLockers, TOTAL_ITEMS);
                std::cout << "[Admin] Lockers Sorted by Availability Status:";
                displayLockerItems(campusLockers, TOTAL_ITEMS);
                break;
            case 3:
```

```
                assetMenu();
                break;
            case 4:
                transactionHistory.showMenu();
                break;
            case 5:
                studentWaitingQueue.processFrontQueue(campusLockers,
TOTAL_ITEMS);
                break;
            case 6:
                studentWaitingQueue.displayQueue();
                break;
            case 7:
                clearScreen();
                break;
            default:
                std::cout << "Invalid Choice." << std::endl;
                break;
        }
    }while(choice != 7);
}

void assetMenu(){
    int choice;
    int id;
    std::string name, status;

    do{
        std::cout << "\n======== CALAMS ASSET INVENTORY ========" << std::endl;
        std::cout << "1. Add Asset" << std::endl;
        std::cout << "2. Search Asset" << std::endl;
        std::cout << "3. Display All Assets" << std::endl;
        std::cout << "4. Update Asset Status" << std::endl;
        std::cout << "5. Exit" << std::endl;
        std:: cout << "Enter your choice: ";
        std:: cin >> choice;

        switch(choice){
            case 1:
                std::cout << "\n===== ADD ASSET =====";
                std::cout << "\nEnter Asset ID: ";
                std::cin >> id;

               std::cin.ignore();

                std::cout << "Enter Asset Name: ";
                getline(std::cin, name);

                std::cout << "Enter Status: ";
                getline(std::cin, status);
```

```
                std::cout << std::endl;

                inventory.addAsset(id, name, status);
                break;

            case 2:
                std::cout << "Enter Asset ID to search: ";
                std::cin >> id;

                inventory.searchAsset(id);
                break;

            case 3:
                inventory.displayAssets();
                break;

            case 4:
                std::cout << "Enter Asset ID: ";
                std::cin >> id;

                std::cin.ignore();

                std::cout << "Enter New Status: "<< std::endl;
                getline(std::cin, status);

                inventory.updateStatus(id, status);
                break;

            case 5:
                break;

            default:
                std::cout << "Invalid choice.";
        }

    } while (choice != 5);
}
```

A.2 Login.h

C/C++

```
#ifndef LOGIN_H
#define LOGIN_H

#include <iostream>
#include <string>
#include "Registration.h"
```

```
const int size = 100;
Registration *accounts = new Registration[size];
int acc_counter = 0;

// Clear Screen Function
void clearScreen() {
    std::cout << "\033[2J\033[3J\033[H" << std::flush;
}

void Register(){
    std::string user;
    std::string pass;
    bool valid;
    do{
        std::cout << "======== Registration =========" << std::endl;
        std::cout << "Username: ";
        std::cin >> user;
        valid = true;
        for(int i = 0; i<acc_counter;i++){
            if(user == accounts[i].getUsername()){
                clearScreen();
                std::cout << "Username is already taken." << std::endl;
                valid = false;
                break;
            }
        }
    }while(!valid);

    std::cout << "Password: ";
    std::cin >> pass;
    accounts[acc_counter] = Registration(user,pass);
    acc_counter++;
    clearScreen();
    std::cout << "Successfully Created Account!" << std::endl;

    return;
}

bool studentLogin(){
    if(acc_counter ==  0){
        std::cout << "No registered accounts yet." << std::endl;
        return false;
    }
    std::string user;
    std::string pass;
    Registration *curr_acc = nullptr;
    int attempts = 3;
    do{
        std::cout << "======== Student Login ========" << std::endl;
        std::cout << "Username: ";
```

```
        std::cin >> user;
        curr_acc = nullptr;
        for(int i = 0; i<acc_counter; i++){
            if(user == accounts[i].getUsername()){
                curr_acc = &accounts[i];
                break;
            }
        }

        if(curr_acc == nullptr){
            std::cout << "Invalid Username." << std::endl;
        }
    }while(curr_acc == nullptr);
    while(attempts != 0){
        std::cout << "Password: ";
        std::cin >> pass;
        if(pass == curr_acc->getPassword()){
            return true;
        }else{
            std::cout << "Incorrect Password." << std::endl;
            attempts--;
            std::cout << "You have " << attempts << " attempts left." <<
std::endl;
        }
    }
    std::cout << "Access Denied." << std::endl;
    exit(0);
}

bool adminLogin(){
    std::string user;
    std::string pass;
    bool valid;
    int attempts = 3;
    do{
        std::cout << "======== Admin Login ========" << std::endl;
        std::cout << "Username: ";
        std::cin >> user;
        valid = false;
        if(user == "admin"){
            valid = true;
        }else{
            clearScreen();
            std::cout << " Invalid Username." << std::endl;
        }
    }while(!valid);
    while(attempts != 0){
        std::cout << "Password: ";
        std::cin >> pass;
        if(pass == "admin"){
```

```
            return true;;
        }else{
            attempts--;
            std::cout << "Incorrect Password." << std::endl;
            std::cout << "You have " << attempts << " attempts left." <<
std::endl;
        }
    }
    std::cout << "Access Denied." << std::endl;
    exit(0);
}

void cleanupAccounts() {
    delete[] accounts;
    accounts = nullptr;
    acc_counter = 0;
}

#endif
```

A.3 Registration.h

C/C++

```
#ifndef REGISTRATION_H
#define REGISTRATION_H

#include <string>

class Registration{
    private:
        std::string username;
        std::string password;
    public:
    //constructor
    Registration(std::string name = "user", std::string pass = "pass"){
        username = name;
        password = pass;
    };

    //destructor
    ~Registration(){
    }

    //Copy Constructor
    Registration(const Registration &copyRegistration){
        username = copyRegistration.username;
        password = copyRegistration.password;
    };
```

```
    //Copy Assignment Operator
    Registration& operator=(const Registration& copyRegistration){
        if(this != &copyRegistration){
            username = copyRegistration.username;
            password = copyRegistration.password;

        }
        return *this;
    };

    std::string getUsername(){
        return username;
    }

    std::string getPassword(){
        return password;
    }
};
#endif
```

A.4 studentQueue.h

C/C++

```
#ifndef STUDENT_QUEUE_H
#define STUDENT_QUEUE_H

#include <iostream>
#include <string>

struct StudentRequest{
    std::string studentName;
    std::string studentID;
    int targetLockerID;
    std::string requestType;
    StudentRequest* next;
};

class StudentQueue{
private:
    StudentRequest* front = nullptr;
    StudentRequest* rear = nullptr;
    int count = 0;

public:
    StudentQueue() = default;
```

```
    ~StudentQueue(){
        while (!isEmpty()){
            dequeue();
        }
    }

    bool isEmpty() const {
            return front == nullptr;
    }

    void enqueue(std::string name, std::string id, int lockerID, std::string
request){
        StudentRequest* newNode = new StudentRequest{name, id, lockerID, request,
nullptr};
        if (isEmpty()){
            front = rear = newNode;
        } else {
            rear->next = newNode;
            rear = newNode;
        }
        count++;
        std::cout << "\n" << name << " (ID: " << id << ") added to the waiting
queue for Locker #" << lockerID << ".\n";
    }

    void dequeue(){
        if (isEmpty()){
            std::cout << "No pending requests in the queue.\n";
            return;
        }
        StudentRequest* temp = front;
        front = front->next;
        if (front == nullptr){
            rear = nullptr;
        }

        std::cout << "\nProcessing locker assignment for: " << temp->studentName
<< " (ID: " << temp->studentID << ")\n";
        delete temp;
        count--;
    }

    // Input and Enqueue
    void requestLockerFromInput(){
        std::string name, id;
        int lockerID;
        std::cout << "\n=== Request / Release a Locker ===\n";
        std::cout << "Enter Student Name: ";
        std::cin.ignore();
        std::getline(std::cin, name);
```

```
        std::cout << "Enter Student ID: ";
        std::cin >> id;
        std::cout << "Enter Target Locker ID: ";
        std::cin >> lockerID;

        enqueue(name, id, lockerID, "Locker Request");
    }

    void processFrontQueue(LockerItem lockers[], int totalLockers) {
        if (isEmpty()) {
            std::cout << "No pending requests in the queue.\n";
            return;
        }

        StudentRequest* req = front;

        for (int i = 0; i < totalLockers; ++i) {
            if (lockers[i].id == req->targetLockerID) {
                // If owned by this student
                if (lockers[i].assignedTo == req->studentID) {
                    lockers[i].isAvailable = true;
                    lockers[i].assignedTo = "Available";
                    std::cout << "\n Locker #" << req->targetLockerID << " is now
released.\n";
                }
                // If available
                else if (lockers[i].isAvailable) {
                    lockers[i].isAvailable = false;
                    lockers[i].assignedTo = req->studentID;
                    std::cout << "\nLocker #" << req->targetLockerID << "
assigned to " << req->studentName << ".\n";
                }
                else {
                    std::cout << "\nLocker #" << req->targetLockerID << " is
occupied by someone else.\n";
                }

                dequeue();
                return;
            }
        }
        // if not found
        std::cout << "\nLocker ID #" << req->targetLockerID << " not found.\n";
        dequeue();
    }

    void displayQueue() const {
        if (isEmpty()){
            std::cout << "\nQueue is currently empty.\n";
            return;
```

```
        }

        StudentRequest* current = front;
        int pos = 1;
        std::cout << "\n--- Locker Request Waiting Line (" << count << " Waiting)
---\n";
        while (current != nullptr){
            std::cout << pos << ". " << current->studentName << " | ID: " <<
current->studentID  << " | Locker #" << current->targetLockerID << "\n";
            current = current->next;
            pos++;
        }
        std::cout << "-----------------------------------------------------\n";
    }
};
#endif
```

A.5 BorrowReturnHistory.h

C/C++

```
#ifndef BORROWRETURNHISTORY_H
#define BORROWRETURNHISTORY_H

#include <iostream>
#include <string>
#include <ctime>
#include <cctype>
#include <limits>

// Settings
const int HISTORY_DEFAULT_CAPACITY = 50;
const int HISTORY_EMPTY = -1;
const int HISTORY_DATE_LENGTH = 32;
const std::string HISTORY_DATE_FORMAT = "%Y-%m-%d %I:%M %p";
const std::string HISTORY_BORROW = "Borrow";
const std::string HISTORY_RETURN = "Return";
const char HISTORY_YES = 'y';
enum HistoryMenu { HISTORY_VIEW_TOP = 1, HISTORY_VIEW_ALL, HISTORY_REMOVE_TOP,
HISTORY_BACK };

// One borrowing or returning transaction
struct BorrowReturnRecord{
    int transactionID;
    std::string username, assetID, type, dateTime;
};

// Array-based stack of recent transactions (LIFO)
class BorrowReturnHistory{
```

```
    private:
        BorrowReturnRecord *records;
        int capacity;
        int topIndex;
        int lastTransactionID;

        std::string getCurrentDateTime(){
            time_t now = time(nullptr);
            char buffer[HISTORY_DATE_LENGTH];
            strftime(buffer, sizeof(buffer), HISTORY_DATE_FORMAT.c_str(),
localtime(&now));
            std::string dateTime = buffer;
            size_t hourStart = dateTime.find(' ') + 1;
            if(dateTime[hourStart] == '0'){ dateTime.erase(hourStart, 1); }
            return dateTime;
        }

        void displayRecord(BorrowReturnRecord record){
            std::cout << "#" << record.transactionID << " | " << record.type << "
| User: " << record.username
                      << " | Asset: " << record.assetID << " | " <<
record.dateTime << std::endl;
        }

    public:
    //constructor
    BorrowReturnHistory(int maxRecords = HISTORY_DEFAULT_CAPACITY){
        capacity = (maxRecords > 0) ? maxRecords : HISTORY_DEFAULT_CAPACITY;
        records = new BorrowReturnRecord[capacity];
        topIndex = HISTORY_EMPTY;
        lastTransactionID = 0;
    }

    //destructor
    ~BorrowReturnHistory(){
        delete[] records;
    }

    // Copying is blocked so two objects never delete the same array
    BorrowReturnHistory(const BorrowReturnHistory &) = delete;
    BorrowReturnHistory& operator=(const BorrowReturnHistory &) = delete;

    bool isEmpty(){ return topIndex == HISTORY_EMPTY; }
    bool isFull(){ return topIndex == capacity - 1; }
    int getCount(){ return topIndex + 1; }

    // Adds a transaction on top
    bool push(std::string username, std::string assetID, std::string type){
        if(isFull()){
            std::cout << "Transaction history is full." << std::endl;
```

```
            return false;
        }
        if(username.empty() || assetID.empty() || (type != HISTORY_BORROW && type
!= HISTORY_RETURN)){
            std::cout << "Invalid transaction. Fill in the username and asset ID,
and use " << HISTORY_BORROW << " or " << HISTORY_RETURN << "." << std::endl;
            return false;
        }
        topIndex++;
        lastTransactionID++;
        records[topIndex] = {lastTransactionID, username, assetID, type,
getCurrentDateTime()};
        return true;
    }

    // Removes the newest entry from the history only
    bool pop(){
        if(isEmpty()){
            std::cout << "No transactions to remove." << std::endl;
            return false;
        }
        topIndex--;
        return true;
    }

    // Returns the newest transaction without removing it
    BorrowReturnRecord top(){
        if(isEmpty()){ return BorrowReturnRecord(); }
        return records[topIndex];
    }

    // Asks for the details, then pushes the transaction
    void recordTransaction(std::string type){
        std::string username, assetID;
        std::cout << "Username: ";
        std::cin >> username;
        std::cout << "Asset ID: ";
        std::cin >> assetID;
        if(push(username, assetID, type)){
            std::cout << type << " recorded." << std::endl;
        }
    }

    bool recordBorrow(std::string username, std::string assetID){ return
push(username, assetID, HISTORY_BORROW); }
    bool recordReturn(std::string username, std::string assetID){ return
push(username, assetID, HISTORY_RETURN); }

    void displayTop(){
```

```
        if(isEmpty()){ std::cout << "No transactions recorded yet." << std::endl;
return; }
        displayRecord(top());
    }

    // Shows newest to oldest
    void displayAll(){
        if(isEmpty()){ std::cout << "No transactions recorded yet." << std::endl;
return; }
        std::cout << getCount() << "/" << capacity << " transactions (newest
first):" << std::endl;
        for(int i = topIndex; i > HISTORY_EMPTY; i--){
            displayRecord(records[i]);
        }
    }

    // Asks before popping
    void confirmPop(){
        if(isEmpty()){ std::cout << "No transactions to remove." << std::endl;
return; }
        displayTop();
        char answer;
        std::cout << "Remove this entry from the history? (y/n): ";
        std::cin >> answer;
        if(std::tolower(answer) == HISTORY_YES && pop()){
            std::cout << "Entry removed. The borrow/return itself is not
cancelled." << std::endl;
        }else{
            std::cout << "Nothing was removed." << std::endl;
        }
    }

    // Admin submenu
    void showMenu(){
        int choice;
        do{
            std::cout << "======== Transaction History ========\n"
                      << HISTORY_VIEW_TOP << ". View Most Recent Transaction\n"
                      << HISTORY_VIEW_ALL << ". View All Recent Transactions\n"
                      << HISTORY_REMOVE_TOP << ". Remove Most Recent
Transaction\n"
                      << HISTORY_BACK << ". Back\n"
                      << "Enter your choice: ";
            // Letters typed: cin sets choice to 0, then the bad input is cleared
            if(!(std::cin >> choice)){
                std::cin.clear();
                std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
            }
            if(choice != HISTORY_BACK){ std::cout << std::endl; }
```

```
            switch(choice){
                case HISTORY_VIEW_TOP: displayTop(); break;
                case HISTORY_VIEW_ALL: displayAll(); break;
                case HISTORY_REMOVE_TOP: confirmPop(); break;
                case HISTORY_BACK: break;
                default: std::cout << "Invalid Choice." << std::endl;
            }
            if(choice != HISTORY_BACK){ std::cout << std::endl; }
        }while(choice != HISTORY_BACK);
    }
};

// Shared transaction history used by the whole system
BorrowReturnHistory transactionHistory;

#endif
```

A.6 LockerAssetSort.h

C/C++

```
#ifndef LOCKER_ASSET_SORT_H
#define LOCKER_ASSET_SORT_H

#include <iostream>
#include <string>
#include <iomanip>

// Structure representing a campus locker or asset entry
struct LockerItem {
    int id;                 // Locker / Asset ID (primary numeric key)
    std::string name;       // Name or description (e.g., "Locker A-101",
"Digital Multimeter")
    std::string assignedTo; // Student number or "Available"
    bool isAvailable;
};

// 1. Selection Sort (By ID - Ascending)
// Performs at most (n - 1) swaps: O(n^2) time, O(1) space
void selectionSortById(LockerItem arr[], int n) {
    for (int i = 0; i < n - 1; ++i) {
        int minIdx = i;
        for (int j = i + 1; j < n; ++j) {
            if (arr[j].id < arr[minIdx].id) {
                minIdx = j;
            }
        }
        if (minIdx != i) {
            LockerItem temp = arr[i];
```

```
            arr[i] = arr[minIdx];
            arr[minIdx] = temp;
        }
    }
}

// 2. Short Bubble Sort (By ID - Ascending)
// Features early termination flag: O(n) best-case, O(n^2) worst-case
void bubbleSortById(LockerItem arr[], int n) {
    bool swapped;
    for (int i = 0; i < n - 1; ++i) {
        swapped = false;
        for (int j = 0; j < n - i - 1; ++j) {
            if (arr[j].id > arr[j + 1].id) {
                LockerItem temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }
        // If no two elements were swapped by inner loop, then array is sorted
        if (!swapped) break;
    }
}

// 3. Selection Sort by Status (Available First)
void sortByAvailability(LockerItem arr[], int n) {
    for (int i = 0; i < n - 1; ++i) {
        int maxIdx = i; // Place available (true) items at the front
        for (int j = i + 1; j < n; ++j) {
            if (arr[j].isAvailable && !arr[maxIdx].isAvailable) {
                maxIdx = j;
            }
        }
        if (maxIdx != i) {
            LockerItem temp = arr[i];
            arr[i] = arr[maxIdx];
            arr[maxIdx] = temp;
        }
    }
}

// Utility function to print tabular list
void displayLockerItems(const LockerItem arr[], int n) {
    std::cout <<
"\n-----------------------------------------------------------------\n";
    std::cout << std::left << std::setw(8)  << "ID"
              << std::setw(25) << "Item / Locker Name"
              << std::setw(15) << "Status"
              << std::setw(15) << "Assigned To" << "\n";
```

```
    std::cout <<
"-----------------------------------------------------------------\n";
    for (int i = 0; i < n; ++i) {
        std::cout << std::left << std::setw(8)  << arr[i].id
                  << std::setw(25) << arr[i].name
                  << std::setw(15) << (arr[i].isAvailable ? "Available" :
"Occupied/Borrowed")
                  << std::setw(15) << arr[i].assignedTo << "\n";
    }
    std::cout <<
"-----------------------------------------------------------------\n";
}

#endif
```

A.7 BST.h

```
C/C++
#ifndef BST_H
#define BST_H

template <typename T>
class BinarySearchTree{
    private:
        //create a key
        T key;

        //pointers for left and right
        BinarySearchTree<T>* left;
        BinarySearchTree<T>* right;
    public:
        //constructor
        BinarySearchTree(T value);

        //destructor
        ~BinarySearchTree();

        //insert left and right
        void insert(T value);

        //traversing
        void preOrder();
        void inOrder();
        void postOrder();
```

```
        //searching
        bool search(T value);

};
template <typename T>
BinarySearchTree<T>::BinarySearchTree(T value){
    key = value;
    left = nullptr;
    right = nullptr;
}

template <typename T>
BinarySearchTree<T>::~BinarySearchTree(){
    delete left;
    delete right;
}
template <typename T>
void BinarySearchTree<T>::insert(T value){
    //left or right

    //go left if value is < key insert to left
    if(value < key){
        if(left == nullptr){
            left = new BinarySearchTree<T>(value);
        }else{
            left->insert(value);
        }
    //go right if value > key insert to right
    }else if(value > key){
        if(right == nullptr){
            right = new BinarySearchTree<T>(value);
        }else{
            right->insert(value);
        }
    }

}

template <typename T>
bool BinarySearchTree<T>::search(T value){
    if (value == key){
        return true;
    }

    if (value < key){
        if (left != nullptr){
            return left->search(value);
        }
    }
```

```
    else{
        if (right != nullptr){
            return right->search(value);
        }
    }

    return false;
}

template <typename T>
void BinarySearchTree<T>::preOrder(){
    std::cout << key << " ";
    if(left != nullptr){
        left->preOrder();
    }

    if(right != nullptr){
        right->preOrder();
    }
}

template <typename T>
void BinarySearchTree<T>::inOrder(){
    if(left != nullptr){
        left->inOrder();
    }
    std::cout << key << " ";
    if(right != nullptr){
        right->inOrder();
    }
}

template <typename T>
void BinarySearchTree<T>::postOrder(){
    if(left != nullptr){
        left->postOrder();
    }
    if(right != nullptr){
        right->postOrder();
    }
    std::cout << key << " ";
}
#endif
```

A.8 AssetInventory.h

C/C++

```
#ifndef ASSETINVENTORY_H
```

```
#define ASSETINVENTORY_H

#include "BST.h"
#include <iostream>
#include <string>

class Asset{
    private:
        int assetID;
        std::string name;
        std::string status;
    public:
        Asset(int id = 0, std::string assetname="", std::string stat =""){
            assetID = id;
            name = assetname;
            status = stat;
        }

        int getAssetID() {
            return assetID;
        }

        std::string getAssetName() {
            return name;
        }

        std::string getStatus() {
            return status;
        }

        void displayAsset() {
            std::cout << "\nAsset ID: " << assetID << std::endl;
            std::cout << "Asset Name: " << name << std::endl;
            std::cout << "Status: " << status << std::endl;
        }

};

class AssetInventory {
private:
    static const int sizeAssets = 100;

    Asset assets[sizeAssets];
    int assetCount;

    BinarySearchTree<int>* assetBST;

public:
    // Constructor
    AssetInventory(){
```

```
        assetCount = 0;
        assetBST = nullptr;;
    }

    // Destructor
    ~AssetInventory() {
        delete assetBST;
    }

    // Add asset
    void addAsset(int id, std::string name, std::string status){
        if (assetBST != nullptr && assetBST->search(id)){
            std::cout << "Asset ID already exists.\n";
            return;
        }

        assets[assetCount] = Asset(id, name, status);
        assetCount++;

        if(assetBST == nullptr){
            assetBST = new BinarySearchTree<int>(id);
        }
        else{
            assetBST->insert(id);
        }

        std::cout << "Asset added successfully!" << std::endl;
    }

    // Search asset
    void searchAsset(int id) {
        if(assetBST == nullptr || !assetBST->search(id)){
            std::cout << "Asset not found.\n";
            return;
        }

        for(int i = 0; i <assetCount;i++){
            if(assets[i].getAssetID() == id){
                std::cout << "===== ASSET FOUND =====";
                assets[i].displayAsset();
                return;
            }
        }
    }

    // Display all assets
    void displayAssets() {
        if(assetCount == 0){
            std::cout << "Asset Inventory Empty." << std::endl;
            return;
```

```
        }

        std::cout << "\n===== ASSET INVENTORY =====" << std::endl;
        assetBST->inOrder();

        std::cout << "\n===== ASSET DETAILS =====" << std::endl;

        for(int i = 0; i < assetCount; i++){
            assets[i].displayAsset();
        }
    }

    // Update asset status
    void updateStatus(int id, std::string newStatus){
        for(int i = 0; i < assetCount; i++) {
            if(assets[i].getAssetID() == id){
                assets[i] = Asset(id, assets[i].getAssetName(), newStatus);
                std::cout << "Asset status updated successfully." << std::endl;
                return;
            }
        }

        std::cout << "Asset not found." << std::endl;
    }
};

#endif
```

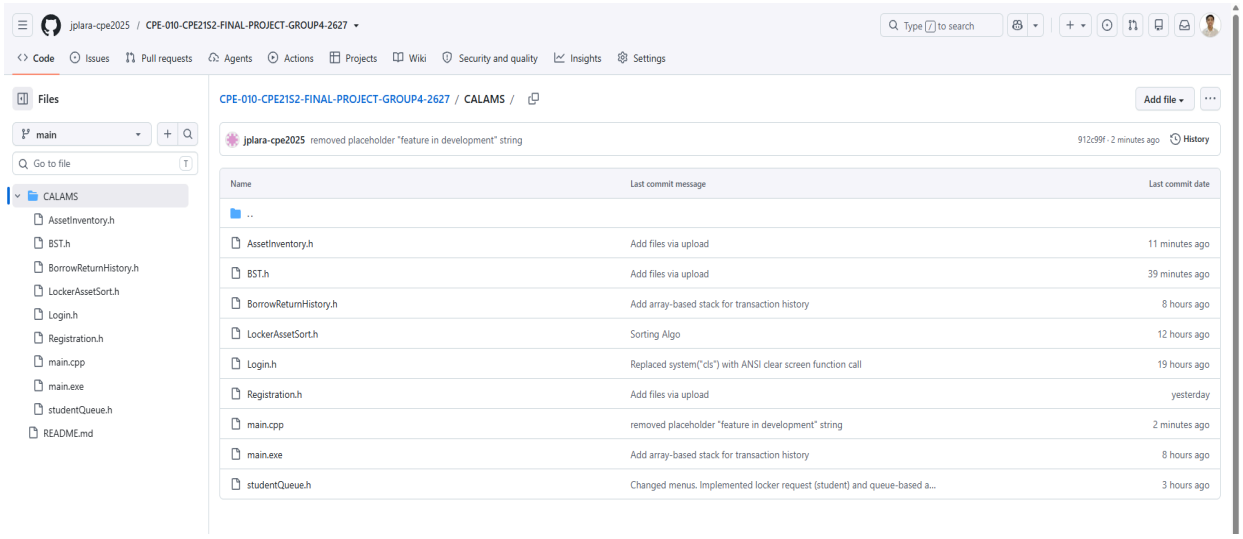
Appendix B. Team Composition

#	Name (Surname, First Name M.I.)	Student No.	Role	Main Contributions	%
1	Bandigan, Christian Lee	2513120	Project Developer	Registration, Login , Asset Inventory and Main module, flowchart	30%
2	Lara, Juan Paulo	2513505	[ ]	[ ]	[ ]
3	Demain, Jaime Luis	2514137	Project Developer	Sorting Algorithm module, Whole Performance Analysis	15%

4	Milan, Kherwin Emmanuel	2513324	Project Developer	Stack (array) module, Current Process (As-Is), Proposed System, 2 Scope and Delimitations	17%
5	Mencidor, Renverlanz	23			
6	Nieles, Cenndy	25			

Appendix C. GitHub Repository

Item	Details
Repository link	https://github.com/jplara-cpe2025/CPE-010-CPE21S2-FINAL-PROJECT-GROUP4-2627.git
Branch / release submitted	main
README contents	[Project description, how to compile and run, sample input, team members]
How to compile	Download all the files in the CALAMS folder and run main.cpp



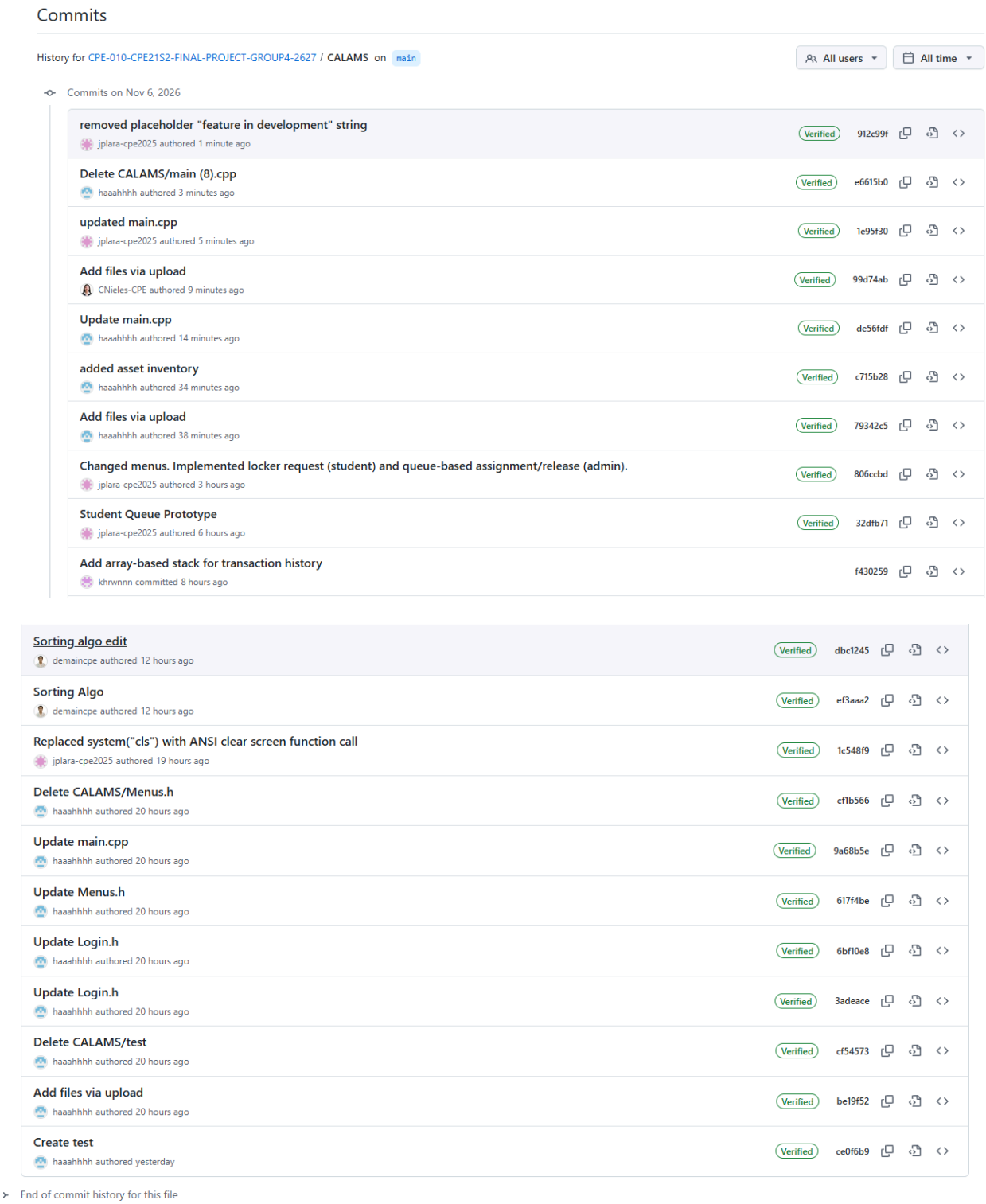


Figure 6. Screenshot of the repository page showing files and commit history

EVALUATION CRITERIA

Criteria	Description	Points
Business Process	Realistic process clearly described; problems and proposed system well defined	15
Data Structures and Algorithms Applied	At least five course topics, correctly explained and justified for the business process	25
System Flowchart	Complete, correct symbols, matches the program	10
Output	All features shown working, including invalid input handling	15
Performance Analysis	Correct theoretical complexity; sound experimental method; meaningful comparison	25
Documentation and Appendices	Complete code, team composition, working GitHub repository; clear writing	10
Total		100